



Compiler Project - Milestone 2

Syntax Analyzer

LL(k) Grammar & Top-Down Parser

Released	Monday, September 21st, 2026	Total Marks	10
Due	Saturday, October 3rd, 2026	Submission	LMS + in-person viva
Deliverable	LL(k) grammar + top-down parser	Viva Venue	IBA City Campus

Task

As the next milestone of your compiler project, you are required to design the **context-free grammar (CFG)** for your proposed programming language, transform it into an **LL(k) grammar**, and implement a **top-down syntax analyzer (parser)** based on that grammar. The parser must consume the **token stream** produced by your lexical analyzer and determine whether the input program is syntactically valid.

What to Submit

1. Complete Grammar Specification

- Submit the grammar for the syntactic constructs supported by your language.
- Clearly identify the **start symbol**.
- Your grammar should cover, as relevant: declarations, assignments, expressions, operator precedence and associativity, conditional statements, loops, functions, parameter lists, blocks, return statements, and any language-specific constructs.

2. LL(k) Grammar

- Transform the grammar so that it is suitable for **predictive top-down parsing**.
- Explicitly state the value of **k** used by your parser. Prefer **LL(1)** where possible.
- Eliminate **left recursion** and perform **left factoring** where necessary.
- Resolve grammar ambiguities that prevent predictive parsing while preserving the intended precedence and associativity of operators.
- Submit both the **original grammar** and the **final transformed LL(k) grammar**. Briefly explain the important transformations.

3. FIRST, FOLLOW & Lookahead Information

- For an LL(1) grammar, provide the relevant **FIRST** and **FOLLOW** sets used to justify predictive choices.
- If $k > 1$, provide the corresponding lookahead information and clearly explain why additional lookahead is required.

Top-Down Parser Implementation

Implement a working **predictive top-down parser** for your language. A hand-written **recursive-descent parser** is recommended; a table-driven predictive parser is also acceptable provided that it follows your submitted LL(k) grammar.

- The parser must operate on the **tokens generated by your lexical analyzer**, not directly on individual source-code characters.
- Parsing decisions must follow the productions and lookahead rules of your submitted grammar.
- A construct accepted by the parser must be supported by the submitted grammar, and vice versa.
- The implementation should not be a collection of ad-hoc token checks unrelated to the grammar.

Token Stream Integration

For example, your lexical analyzer may produce a token stream such as:

```
<data_type, int>
<identifier, count>
<assignment_operator, =>
<integer_literal, 10>
<semicolon, ;>
```

The syntax analyzer should consume this token stream and determine whether it forms a valid construct according to your LL(k) grammar.

Syntax Error Handling

Your parser must detect syntax errors and **report them clearly**. As far as reasonably possible, an error message should identify **where the error occurred**, **what was expected**, and **what token was actually found**.

```
Syntax Error at line 7:
Expected ")" but found ";"
```

Meaningful messages such as `Expected expression after assignment operator` are preferred over generic messages such as `Invalid syntax`.

Test Programs

Submit test cases that exercise multiple syntactic constructs of your language and include the parser output for each test:

- **At least two syntactically valid programs.**
- **At least two syntactically invalid programs.**

Submission Checklist

Before uploading to the LMS, confirm that your submission includes:

- Original CFG.
- Final transformed LL(k) grammar.
- The value of k used by your parser.
- A brief record of grammar transformations, including left-recursion removal and left factoring where applicable.
- Relevant FIRST and FOLLOW sets, or the required LL(k) lookahead information.
- Source code for the top-down parser.
- Lexical analyzer / token definitions required to run the parser.
- At least two valid and two invalid test programs.
- Parser output for all submitted test programs.
- All files required to compile and run the submitted parser.

Viva - Saturday, October 3rd, 2026

An **in-person viva will be conducted at IBA City Campus on Saturday, October 3rd, 2026.**

All group members must be prepared to explain and defend their own work, including:

- the original grammar and the transformations used to obtain the LL(k) grammar;
- the selected value of k and the parser's lookahead decisions;
- FIRST/FOLLOW information and predictive parsing choices;
- how the parser consumes tokens from the lexical analyzer;
- the implementation of major parser routines / parsing-table entries; and
- the behavior of the parser on valid and invalid input.

Important

The grammar, token definitions, lexical analyzer, and parser must all correspond to the same proposed programming language. Your implementation must genuinely use a top-down parsing approach based on the LL(k) grammar you submit.

Submit all required files on the LMS by **Saturday, October 3rd, 2026** and appear for the viva at **IBA City Campus**.

- End of Assignment -